Final Technical Report

MacraScript: A Domain-Specific Language
for Macramé Pattern Generation

Juan Manuel Serrano Rodriguez - 20211020091
Javier Alejandro Sánchez Salamanca - 20201020167
Computer Science 3

Universidad Distrital Francisco José de Caldas

May 12, 2025

Contents

Abstract

This technical report details the design, implementation, and final results of the MacraScript project. MacraScript is a domain-specific language (DSL) created to address the lack of specialized digital tools for designing and visualizing macramé patterns. The project involved the development of a complete compiler in Python, featuring lexical, syntactic, and semantic analyzers, along with a Tkinter-based graphical user interface. The language supports two distinct pattern types, "Alpha" and "Normal," and generates dual graphical outputs for each: a top-down pixel-art preview and a detailed, step-by-step knot-flow diagram. This document covers the final language grammar, the complete architecture of the compiler, the logic behind the graphical rendering, and the results achieved, demonstrating a functional and effective solution that bridges the gap between traditional crafting and computational design.

Chapter 1

Introduction

1.1 Problem Context

Macramé is a textile art form based on creating decorative patterns through organized knotting sequences. The planning of complex patterns is a meticulous process, prone to human error and lacking specialized digital tools. Artisans often rely on hand-drawn grids or generic design software, which are not optimized for the specific constraints of knot-based patterns [3].

1.2 Project Objectives

The primary goal of this project was to develop MacraScript, a DSL that allows designers to:

- Define macramé patterns using a simple, declarative syntax.
- Specify parameters such as thread count, colors, and knot sequences.
- Generate accurate visualizations of the final pattern.
- Produce step-by-step graphical guides for executing the pattern.

Chapter 2

Final Language Design

2.1 Overview

MacraScript is a declarative DSL designed to be intuitive for macramé artisans. The language is structured around a central `PATTERN` block, which can be of type `ALPHA` (for grid-based designs) or `NORMAL` (for knot-based geometric patterns).

2.2 Final Grammar

The final grammar for MacraScript, defined in EBNF, is as follows:

```
<program> ::= "START" <type> <config> <pattern> "END"

<type> ::= "ALPHA" | "NORMAL"

<config> ::= <threads> [<dims>] <colors>
<threads> ::= "THREADS" ":" <INTEGER>
<dims> ::= ("WIDTH" | "HEIGHT") ":" <INTEGER>
<colors> ::= "COLORS" ":" "(" <string_list> ")"
<string_list> ::= <STRING> ("," <STRING>)*

<pattern> ::= "PATTERN" "{" <row_list> "}"
<row_list> ::= <row>+
<row> ::= "ROW" ":" "(" <sequence> ")"

<sequence> ::= <alpha_sequence> | <normal_sequence>
<alpha_sequence> ::= <INTEGER> ("," <INTEGER>)*
<normal_sequence> ::= <DIRECTION> ("," <DIRECTION>)*
<DIRECTION> ::= "LEFT" | "RIGHT"
```

Listing 2.1: Final EBNF Grammar for MacraScript

2.3 Syntax Examples

2.3.1 Alpha Pattern Example

```
START ALPHA
THREADS: 4
WIDTH: 4
```

```
HEIGHT: 4
COLORS: ("red", "blue")
PATTERN {
    ROW: (0, 1, 0, 1)
    ROW: (1, 0, 1, 0)
    ROW: (0, 1, 0, 1)
    ROW: (1, 0, 1, 0)
}
END
```

Listing 2.2: Alpha pattern for a 4x4 checkerboard

2.3.2 Normal Pattern Example

The syntax for Normal patterns was refined to be more intuitive. Instead of specifying individual knots, the user defines rows of knot directions. The compiler automatically handles the staggered pairing of threads.

```
START NORMAL
THREADS: 8
COLORS: ("red", "blue", "yellow", "green")
PATTERN {
    ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
    ROW: (LEFT, LEFT, LEFT)
    ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
}
END
```

Listing 2.3: Normal pattern with alternating rows

Chapter 3

System Architecture and Implementation

3.1 Compiler Pipeline

The MacraScript compiler is a Python application that processes the source code in three main phases:

1. **Lexical Analysis (`lexical.py`):** This module uses regular expressions to break the input code into a list of tokens. It correctly handles keywords, identifiers, color names, and hex color codes.
2. **Syntactic Analysis (`sintactic.py`):** A recursive descent parser validates the token sequence against the defined grammar. It builds a dictionary representing the abstract structure of the code, which is passed to the next phase.
3. **Semantic Analysis (`semantic.py`):** This is the core of the compiler's logic. It takes the parsed structure and:
    - Validates semantic rules (e.g., thread counts, color indices).
    - For `NORMAL` patterns, it translates the abstract rows of directions into a concrete list of knot instructions, automatically calculating the correct staggered thread pairs for each knot.
    - Produces a high-level `MacraPattern` object that is easy for the rendering engine to consume.

3.2 Graphical User Interface (`compiler.py`)

The main application module uses Tkinter to provide a simple GUI. It includes a text editor for writing MacraScript code and two canvas widgets for displaying the graphical outputs. This module orchestrates the compilation pipeline and calls the appropriate drawing functions.

Chapter 4

Results and Discussion

4.1 Graphical Outputs

The final implementation successfully generates two distinct and coherent visualizations for each pattern type, providing both a final preview and a functional guide.

4.1.1 Pixel Art Visualization

This view shows a top-down preview of the finished piece.

- **Alpha:** Renders a grid of colored squares.
- **Normal:** Renders a grid of colored diamonds. The color of each diamond correctly represents the color of the *active* thread that forms the knot, ensuring visual consistency with the knot diagram.

4.1.2 Knot-Flow Diagram

This view provides a step-by-step guide for the artisan.

- **Alpha:** Renders a snake-like flow of nodes. The connecting lines are colored to match the node they originate from, improving clarity.
- **Normal:** Renders a detailed lattice diagram showing the path of each thread. The knots are represented by circles, colored according to the active thread, with an arrow indicating the knot's direction.

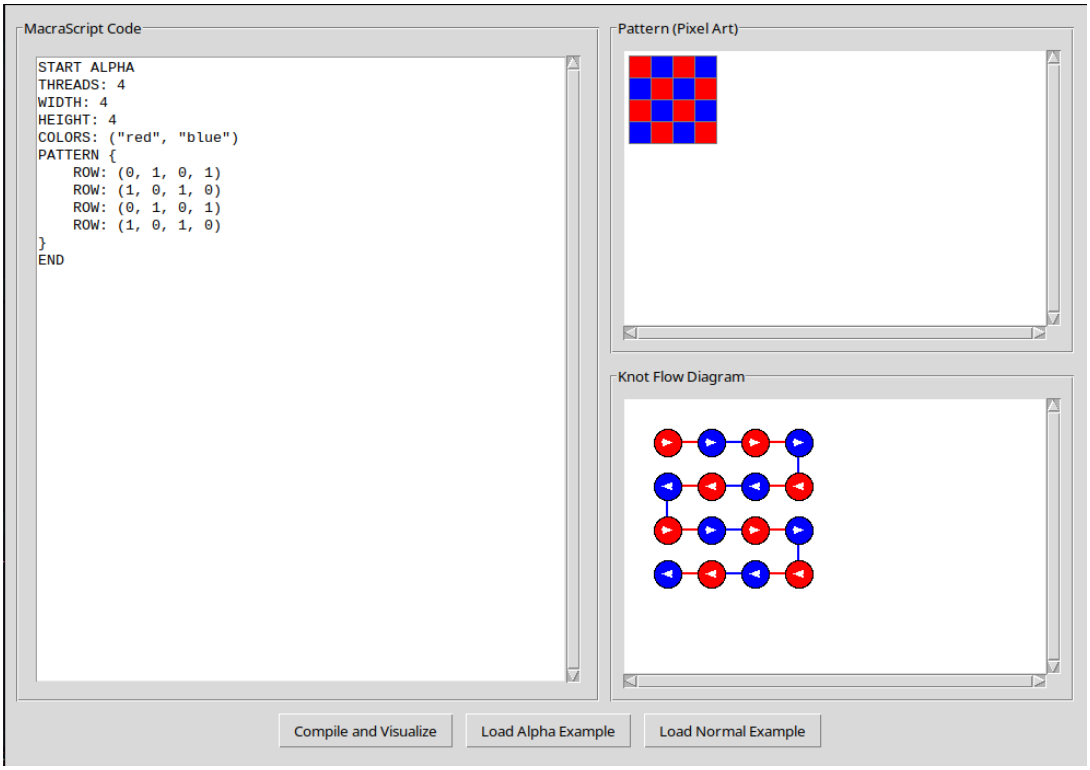

Figure 4.1: Final dual visualization for an Alpha pattern, showing the pixel-art preview and the snake-like knot-flow diagram.

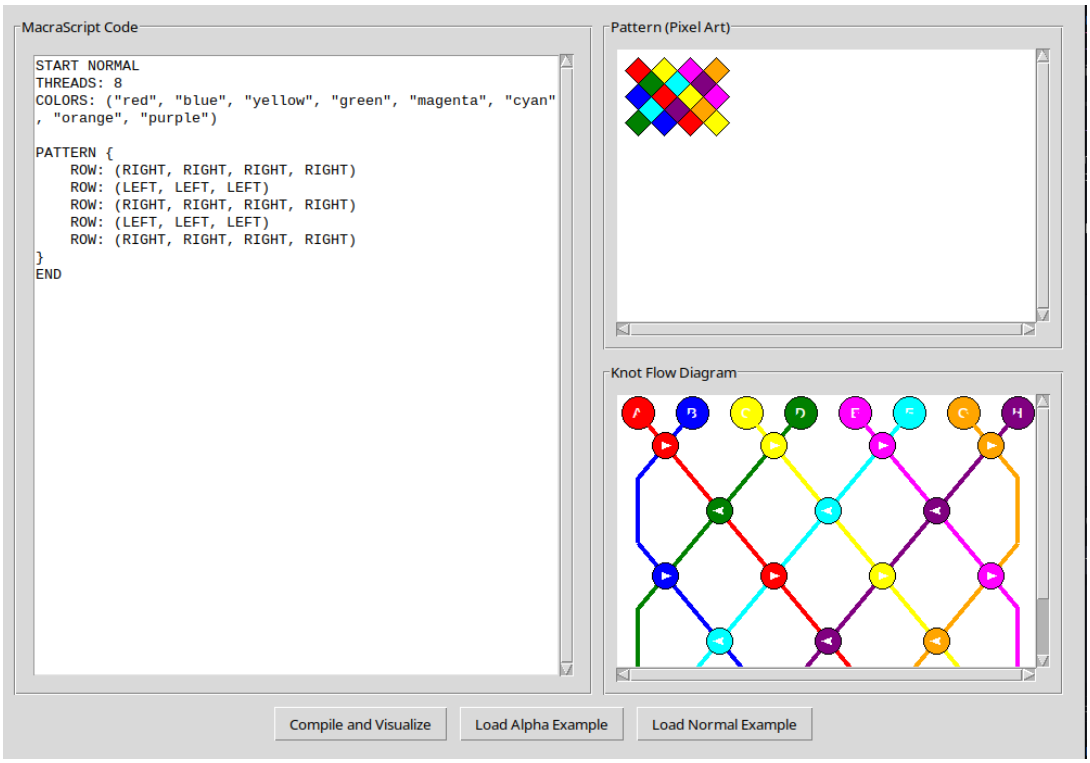

Figure 4.2: Final dual visualization for a Normal pattern, showing the diamond preview and the detailed knot-flow diagram.

4.2 Conclusion

The MacraScript project has successfully achieved its objectives. We have designed and implemented a functional DSL and compiler that significantly simplifies the process of creating and visualizing complex macramé patterns. The final system provides a robust tool that bridges the gap between abstract design and the physical craft, demonstrating the power of computational thinking in a non-traditional domain. The refined language syntax for Normal patterns proved to be a critical improvement, making the language more intuitive and powerful.

Bibliography

[1] Fowler, M. (2010). Domain-Specific Languages. Addison-Wesley Professional.

[2] Mernik, M., Heering, J., & Sloane, A. M. (2005). When and how to develop domain-specific languages. ACM Computing Surveys (CSUR), 37(4), 316-344.

[3] Karner, C. (2005). Macramé: The craft of knotting. Schiffer Publishing.